Trust and signing (authentication)

TBK SPEC 09:2026, Version 1.0

The Tebako Project

Unrestricted
November 01, 2026
TBK SPEC 09:2026, Version 1.0
Tebako
© 2026 The Tebako Project. All rights reserved

Ribose

Contents

Foreword 4

1. Threat model 5

2. The chain of trust 5

3. Opt-in rules (locked) 5

4. Verification points 6

5. Release index authentication 7

6. Tooling 8

7. Ceremony status (roadmap 10) 8

8. Revocation (locked 2026-07-27) 9

9. Third-party identity and key distribution 9

10. Key retrieval from the trust-anchor channel (locked 2026-09-18) 12

© 2026 The Tebako Project

All rights reserved. Unless otherwise specified, no part of this publication may be reproduced or utilized otherwise in any form or by any means, electronic or mechanical, including photocopying, or posting on the internet or an intranet, without prior written permission. Permission can be requested from the address below.

The Tebako Project

Suite 1, 8/F, New Henry House
10 Ice House Street
Central
Hong Kong

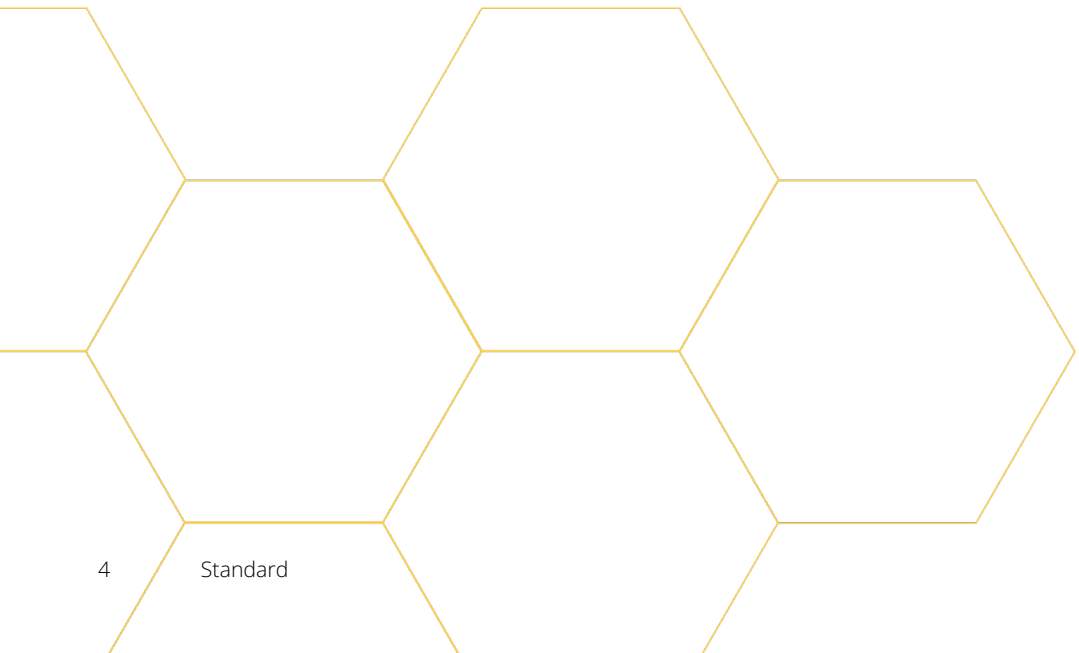
copyright@ribose.com
www.ribose.com

© 2026 The Tebako Project

Foreword

Normative specification of integrity and authenticity for every binary part: bootstrap, runtime payloads, data payloads, and release indexes. Status: signing machinery SHIPPED (M29 phase 2); the classical root key ceremony EXECUTED 2026-09-09 (§7); release signing SHIPPED — each platform leg's publish invocation signs its own released parts and index shards behind `TEBAKO_RELEASE_SIGNING_ENABLED` (§6; the finalize-pass sign step was retired by spec 13 §2a's de-rendezvous, 2026-09-12). Runtime FETCH-TIME verification (§4's resolve-path point, §5's signed index forms, spec 13 §2a's per-artifact `signature`) is SHIPPED in the CLI's press-time resolver and the shim's dispatch-time fetch (roadmap 80/G1, tebako#567's chain); the bootstrap's first-run runtime download is the remaining leg (PLANNED — until it lands, the bootstrap verifies sha256 only and never reads `signature` declarations: the pre-G1 behavior, landing together with the `openpgp-verify` capability flip of the rollout note below). Payload verification (§4's install-time point, extended 2026-09-13 to the press-time compose-closure and spawn-walk payload fetches) is SHIPPED; the store-side anchor stays the `.sha256` sidecar.

> **Rollout phase — unverified-first (roadmap 72).** The shipped tebako-bootstrap is built WITHOUT OpenPGP verification (the `openpgp-verify` cargo feature is off): signed packages execute with a loud UNVERIFIED warning + audit journal, unsigned packages with the legacy warning; `TEBAKO_REQUIRE_SIGNED=1` fails closed (exit 71) because a strict-mode request must never silently downgrade. OpenPGP verification returns via the crypto toolkit payload (`tebako-crypto`, rnp+botan full+PQC, fetched on demand and Ed25519-verified by the bootstrap) — this spec is the TARGET model; the phase-1 deltas live in spec 06 §3.



© 2026 The Tebako Project

1. Threat model

Parts travel over mirrors/CDN and are shared between machines. HTTPS + sha256 against a same-channel manifest protects the TRANSPORT, not the OBJECT: whoever swaps the artifact swaps the manifest too. The tpkg crc32 is integrity-vs-accident only — documented as non-authentic (spec 02 §2). For the relationship to the OS-level trust gates (Gatekeeper, Authenticode) — who validates the bootstrap versus what tebako validates below it — see spec 12 §5.

2. The chain of trust

1. **Root of trust:** the tamatebako release signing keypair (OpenPGP via rnp-rs — the ONE signature mechanism for trailers, manifests, and release indexes alike: one keyring, one verify path). Root fingerprint published on tebako.org AND embedded in the loader/CLI at release time (`EMBEDDED_ROOT_FINGERPRINT`; dev override `TEBAKO_TRUSTED_ROOT`). The CLI additionally embeds the root PUBLIC KEY (`tebako_signer::ROOT_PUBLIC_KEY`, byte-identical with `https://www.tebako.org/.well-known/tebako-key.asc`), so first-party signature verification needs no key distribution at all (§9's zero-interaction rule); the size-gated bootstrap stays fingerprint-only, the public key reaching its keyrings through the normal channels.
2. **Production (releases):** every released part (runtime payloads, bootstrap, src tarballs, libraries) ships a detached `.asc` per artifact PLUS a signed release manifest — one verify path for all.
3. **Packages:** the tpkg v2 extension (spec 02 §4) carries per-slot SHA-256 + signer keyid + the OpenPGP signature over the canonical trailer bytes.
4. **Rotation:** a signed successor-key statement (`TEBAKO-ROOT-SUCCESSOR-V1` — canonical lines binding predecessor → successor fingerprint, detached signature from the predecessor) lets any machine forward trust with no out-of-band step: the loader applies the statement chain from a trusted root and TOFU-registers the successor. Revocation procedure documented and rehearsed.

3. Opt-in rules (locked)

- Signing is **per-package opt-in**: `tebako-pkg bundle --sign[=keyid]`. Without it the package is plain v1, byte-identical to pre-signing — no key, no prompt, no ceremony. A

© 2026 The Tebako Project

press-local key is generated and cached under `$TEBAKO_HOME/keys` only on the first explicit `--sign`.

- Rewrite operations (insert-image/remove-image/set-runtime/reassemble) preserve the input's signing state.
- Only the presence of a signature is optional — verification of signed packages is always strict (exits 71/72, spec 06 §4).
- **v1-legacy rule:** unsigned v1 packages exist in the wild and stay runnable — accepted as LEGACY with a loud stderr warning **on first acceptance per machine** (marker `$TEBAKO_HOME/warned-legacy/<sha256(path|size|mtime)>`; a re-pressed or replaced binary warns again) plus an audit-journal entry on **every** acceptance (tebako#400: the per-run repetition was unactionable noise; the durable remedy is `tebako press --sign` — v2-signed trailers never warn). A package whose developer declared `quiet_notices` (spec 23 §14) carries `TPKG_FLAG_QUIET_NOTICES` (bit 3) and suppresses this warning and the spec 06 §5 progress lines on every run — the bit rides inside the signed region, so it cannot be forged onto a signed package, and the journal entry is written regardless. `TEBAKO_REQUIRE_SIGNED=1` is the explicit opt-in hard-fail for hardened environments — never the default; it is checked FIRST and outranks the quiet bit.
- **Fail closed everywhere:** no insecure-skip flag.
- Encryption is likewise per-image opt-in (spec 10).

4. Verification points

- **Press time:** the CLI verifies release signatures before using any part (fail closed). Trusted keyring in `$TEBAKO_HOME`; our root key embedded (the fingerprint in the loader, the full public key in the CLI); additional keys TOFU-registered with a named prompt. The point covers every part a press CONSUMES: the runtime pair (the runtime-fetch point's press leg, above) and every registry payload the compose closure (spec 23 §13) or the spawn walk (spec 23 §13.6, spec 32 §6) fetches — a declared `signature` verifies BEFORE the bytes enter the payload cache and before the press pins their digest into the lock, so a tampered slice can never be laundered into a lock pin that downstream installs and runs then trust. The matrix is the install-time point's, unchanged: signed verifies strict (71 invalid / 72 untrusted / 72 signer-key-changed); unsigned warns loud + journals (`event=legacy-unsigned-accepted`) and fails closed (71) under `TEBAKO_REQUIRE_SIGNED=1`; a cache hit stands on its trust anchor (spec 05 §4 — verified when the bytes entered, never re-verified per use). This is spec 23 §13.4's trust uniformity made literal: carried and shared slices pass the same gate at the same moment.

© 2026 The Tebako Project

- **Install time:** registry-pinned payload signatures verify against the trusted keyring PLUS the embedded root public key — a tamatebako-signed payload verifies Trusted on a fresh machine, no registration step.
- **Runtime fetch (resolve time — the per-binary fetch paths: the shim's dispatch-time download and the CLI's press-time resolver SHIPPED (roadmap 80/G1); the bootstrap's runtime download is the remaining leg, PLANNED):** the resolver verifies the consumed index form's detached signature (§5) BEFORE trusting its digests, then verifies each fetched artifact whose entry declares `signature` (spec 13 §2a) — always strict: an invalid signature → exit 71; a signer keyid whose primary is not in the trusted keyring → exit 72; a declared `.asc` that does not fetch → exit 71 (a signing declaration without its signature asset is an invalid signing state, never a skip). The sha256 check then runs as today (exit 70). An entry with NO signature anywhere (a pre-signing release line — those releases are keep-forever, spec 13 §8) is accepted with a loud stderr warning + audit journal (`event=unsigned-runtime-fetch`) on every fetch — the download moment IS the trust decision here (§9's two-models rule) — and refused with exit 71 under `TEBAKO_REQUIRE_SIGNED=1`. A bootstrap built without `openpgp-verify` treats declared runtime signatures exactly as it treats signed trailers (spec 06 §3): loud UNVERIFIED warning + journal, sha256 enforced as integrity-only, exit 71 under `TEBAKO_REQUIRE_SIGNED=1` naming the missing capability. The store markers keep their spec 05 §4 semantics — verification happens at fetch/install, never per run.
- **First run:** the loader verifies the trailer signature against the keyring, then each slot's sha256 before mounting/extracting (streaming, one pass at install time; the trust-anchor marker avoids re-hashing every run — spec 05 §4).
- **Runtime driver:** re-verifies an image's sha256 against the trailer before mount (integrity; authenticity already established by the trailer signature).

5. Release index authentication

Every index form a resolver can consume ships a detached `.asc` from the factory's release key, and the resolver verifies the signature of the form it consumed BEFORE trusting its hashes — closing the gap where a MITM swaps both package and unsigned manifest. Same keyring, same verify path. Per runtime factory release (spec 13 §2a): each per-package shard `<stem>.manifest.json` ships `<stem>.manifest.json.asc` (the sidecar-era authority is signed first), every per-asset `<asset>.sha256` sidecar ships `<asset>.sha256.asc`, and every payload asset (the interpreter exe, the env image, a windows dll) ships its own `<asset>.asc`. **No artifact is ever "covered by" another artifact's signature** (2026-09-12, roadmap 85): every served name carries its own `.asc` on signing-enabled lines, signed by the invocation holding the fresh bytes. **Bundle-era lines (additive, 2026-09-24, spec 36) keep the rule exactly**: the served names are the bundle, its sidecar, and the shard — three `.ascs` per leg — and the bundle's MEMBERS are not served names; their authenticity flows from the signed shard's per-member pins verified

© 2026 The Tebako Project

post-unpack (spec 36 §4), never from an assumption about the container. The monoliths are never release assets (spec 13 §2a's de-rendezvous), so no `manifest.json.asc` / `SHA256SUMS.txt.asc` exist on a release; a consumer-side derived index verifies by re-derivation from the signed shards. The entry's `signature: {keyid, asc}` fields name the per-artifact sidecars — the same SSOT rule as `filename`: the factory declares the spellings, consumers flow them verbatim and never synthesize. `keyid` names the signer's PRIMARY keyid (§9's primary-vs-subkey rule). All of it is additive: pre-signing releases carry none of it and stay installable under §4's unsigned-fetch rule (keep-forever, spec 13 §8); a release that declares a `signature` for an artifact MUST ship that `.asc` — a declaration without its signature asset is an invalid signing state (§4: exit 71 at fetch, and the factory's publish fails before that, spec 13 §2a).

6. Tooling

`tebako-pkg sign` (detached `.asc` per artifact + signed `SHA256SUMS`) and `tebako-pkg verify` (Trusted/Untrusted/Invalid per artifact). Factory release flows invoke `tebako-pkg sign` in CI (secrets-held armored root export; the private key never leaves CI secrets/hardware).

7. Ceremony status (roadmap 10)

The production root key ceremony was executed 2026-09-09: the classical Ed25519 root `9E210CA8E9FDE9E6587740B2EFC3C250F7862A48` exists in the offline store, `EMBEDDED_ROOT_FINGERPRINT` is filled, the fingerprint and public key are published on tebako.org (the trust-anchor page and `/.well-known/tebako-key.asc`), the CI secrets are wired (the finalize job signs releases behind `TEBAKO_RELEASE_SIGNING_ENABLED`), and the sign/verify drill was rehearsed against a released tebako-pkg before go-live (trusted and tamper-invalid both proven).

Recorded deviations from the `docs/root-ceremony.md` runbook (roadmap 36), each with its follow-up: no PQC root and no pre-made successor statement yet (the rnp line in use lacks ML-DSA — lands with the tebako-crypto toolkit, roadmap 72); the CI signing-subkey export is unprotected (tebako-signer passphrase-unlock follow-up); software storage in place of hardware tokens; the encryption subkey is X25519 in place of day-one ML-KEM; the revocation certificate is the tool-generated self-revocation. The full deviation record lives with the offline key store.

© 2026 The Tebako Project

8. Revocation (locked 2026-07-27)

Rotation (successor statement) moves trust forward; REVOCATION kills a key with no successor — for compromise or retirement:

FIGURE 1

```
-----BEGIN TEBAKO REVOCATION STATEMENT-----
format: TEBAKO-ROOT-REVOCATION-V1
fingerprint: <40-hex revoked root>
created: <unix seconds>
reason: compromise | retired | superseded
-----BEGIN PGP SIGNATURE-----
<detached signature from the REVOKED key itself (self-
revocation)
 — or from the surviving PREDECESSOR root in a chain>
```

- A self-revocation is proof of possession; a predecessor-signed revocation covers a lost key (the predecessor authorized the chain).
- Consumers apply revocations from the successors dir alongside rotation statements: a revoked fingerprint is rejected even if it previously verified (fail-closed, named RevokedKey → exit 72).
- Revocations are additive files (`<fingerprint>.revoke.asc`), never edited history; the audit journal records every application.
- The ceremony doc (roadmap 36) includes rehearsing this once.

9. Third-party identity and key distribution

First-party slices verify against the embedded tamatebako root (§2). Third-party authors are not tamatebako; their slices and fat/slim binaries verify against THEIR OWN keys, and the question that decides everything is how a user's machine learns the right fingerprint. The method is the three-channel pattern — no channel alone is trusted, two channels agreeing are the trust event:

1. **The registry (location-as-identity, primary channel).** A registry ref pins an authenticated location (`tfs:github:metanorma/metanorma` is GitHub's guarantee that this is the metanorma org). The registry index carries a `signing:` block at its

© 2026 The Tebako Project

head: armored public key, fingerprint, and an optional canonical key URL. `tebako add-registry` fetches the index, verifies the block's key IS its declared fingerprint (an inconsistent block is the registry's malformation, exit 65 — never a trust question), displays the fingerprint and the cross-check URLs, and asks for confirmation (TOFU). On confirmation the key is pinned to `$TEBAKO_HOME/trust/<FINGERPRINT>.pub` and the registry's config book entry records the fingerprint (`signing_fingerprint:`) so re-adds short-circuit and `tebako keys list` shows the binding [shipped — tebako#617]. An already-trusted key (embedded root, keyring, an earlier pin) asks nothing. Non-interactive flows supply the expected fingerprint out of band via `--expect-fingerprint <hex>` — never a blind yes; a mismatch is the trust class (exit 72) printing both fingerprints.

2. **Out-of-band confirmation (second channel, never auto-trusted).** Authors publish the same fingerprint where their audience already trusts them: `https://<author-domain>/.well-known/tebako-key.asc` (the well-known convention), their docs/README, and for GitHub-hosted projects the org's `.github` repository (zero new infrastructure). The CLI prints these URLs for the user to compare; it never upgrades trust from the second channel silently.
3. **Fingerprint-addressed directories (no trust needed).** OpenPGP keyservers are used as directories: fetch BY fingerprint, authenticate the fetched key against the channels above. A swapped key has the wrong fingerprint by construction, so the directory itself needs no trust.

Continuity is the security property. After the first confirmation, every artifact from that identity must verify against the pinned key. A key mismatch fails closed (`SignerKeyChanged`, exit 72) and displays both fingerprints; a valid successor chain (§8 rotation) forwards automatically after displaying the chain proof, re-pinning the new key.

A signature pin names the PRIMARY keyid — the identity, not the instrument. A well-formed release key signs data with a signing SUBKEY (a certify-only primary never signs; the 2026-09-09 root is this shape), so the signature's issuer keyid is the subkey's, while the identity a registry pins is the primary's (its low 64 bits, 16 lowercase hex). Verification therefore resolves the verified issuer to its primary through the keyring before comparing (`tebako_signer::primary_keyid_of`); a pin naming the issuer subkey directly also matches (exact compare first). Consequence: rotating the signing SUBKEY under an unchanged primary never invalidates existing registry pins. Publishers pin the primary — `tebako-pkg sign` and `tebako publish` both report it (the sign line names the primary keyid; when a subkey did the signing, the issuer is reported alongside for transparency).

Per artifact class:

- Registry index: verified against the pinned registry key on every refresh.
- Payload slices: signed trailer verified against the author's pinned key; per-slot digests chain off it (§4).
- Runtime slices (tamatebako): the embedded root — nothing to register.
- Runtime slices (third-party — discovered through a registry's `kind: runtime` entry, spec 05 §2's registry-derived base): the publishing registry's pinned key — the channel that supplied the download base also anchors the key, so a third-party runtime adds

© 2026 The Tebako Project

no ceremony past the add-registry TOFU. The entry's `signature.keyid` must resolve to that pinned primary (mismatch → `SignerKeyChanged`, exit 72).

- Third-party fat/slim binaries: verified at install/import against the pinned author key (the same verify path); the author may ALSO OS-codesign with their own Developer ID — Apple/Microsoft gates then vouch at execution time, orthogonally and complementarily (spec 12 §5).

The two distribution models decide what a warning is worth (tebako#400). Who made the trust decision, and when, differs by package shape — the notice policy follows from it:

- **Fat binary (bootstrap + payload downloaded as ONE artifact).** The user's trust decision is explicit and happens ONCE, at download time — fetching the binary from a location they chose, like any binary download. A per-run signature warning on such a package is unactionable noise: the user cannot re-verify at run time what they already decided to fetch and run. The developer may therefore declaratively suppress it for every run of the package (`quiet_notices`, spec 23 §14 — shipped; the bit rides inside the signed region, and `TEBAKO_REQUIRE_SIGNED=1` still outranks it).
- **Separately-resolved lean payloads.** The package names its runtime and slices; the loader RESOLVES them at first run from whatever the registries serve at that moment. The download-time decision no longer covers the payload bytes — here the signature IS the trust mechanism (§4's verification points), and the warnings on unsigned resolutions stay justified and loud.

Press signing (shipped — tebako#400 S1). Signing stays per-package OPT-IN (§3): a press with no signing declaration on any channel produces the v1-unsigned trailer, byte-identical to pre-signing output, and never touches key material. The declaration rides the `sign` registry setting (spec 23 §14) through three channels, resolved **CLI → env → compose document → default (unsigned)** — auto-sign must never surprise:

- `tebako press --sign` signs the produced package with the press-local key (generated and cached under `$TEBAKO_HOME/keys` on first explicit use, auto-registered into the local trusted keyring); `--sign=<keyid>` signs with the named secret key from `$TEBAKO_HOME/keys` — a keyid matching no key is a NAMED error raised before any heavy work, never a silent fallback to the default key.
- `TEBAKO_SIGN=1` is the machine-wide declaration and the compose document's `sign: true` the repo-carried one; both sign with the press-local key (a keyid is per-machine material and never rides a git-shared document).
- `--no-sign` (or `TEBAKO_SIGN=0`) wins over every lower channel. An opt-out that overrides a lower channel's `sign` declaration is LOUD — a stderr warning plus an audit-journal entry (`event=press-sign-opt-out`) — because silently dropping an authored signature declaration would be a quiet trust downgrade (invariant 9).

Developer ergonomics (target UX; status in braces). `tebako keygen` creates a keypair in `$TEBAKO_HOME/keys/` and prints the fingerprint with the exact text to paste into docs + the well-known location [partial: key creation exists]; `tebako press --sign[=<keyid>]` signs the produced package, with `TEBAKO_SIGN` / the compose `sign:` key as the machine-wide and repo-carried declarations and `--no-sign` as the loud opt-out

© 2026 The Tebako Project

[shipped]; `tebako publish` signs every artifact + the index and writes the `signing:` block into the registry index automatically [shipped in install-UX]; `tebako key rotate` creates + signs + publishes a successor statement in one command [planned]; revocation follows §8 [format locked].

User ergonomics. First-party: zero interaction, ever (embedded root — the CLI carries the root public key, so first-party artifacts verify Trusted on a fresh machine). Third-party: one informed consent per author (the TOFU prompt), then silent verification forever. `tebako keys list|remove` manage the signing-key trust inputs (the keyring and the add-registry pin store) [shipped — tebako#617; the `trust` verb family is the TLS CA store, spec 22 §4]; `tebako doctor` re-verifies installed artifacts against pinned keys [roadmap 50]. Unverified artifacts always produce the loud warning + journal (§3); `TEBAKO_REQUIRE_SIGNED=1` refuses them outright.

10. Key retrieval from the trust-anchor channel (locked 2026-09-18)

A signed artifact whose pinned `signature.keyid` is absent from the local keyring used to be a dead end ("register the publisher's key, then retry"). For keys ON THE TAMATEBAKO ROOT CHAIN the machine can now close that gap itself: the pubkey is fetched from the trust-anchor publication (§2's tebako.org channel) and admitted ONLY through the chain — trust never extends silently, and never extends by TOFU.

- The well-known grammar (the tebako.org publication owns these spellings; consumers flow them, never re-derive):
  - `https://www.tebako.org/.well-known/tebako-keys/<keyid>.asc` — the armored public key whose PRIMARY keyid (16 lowercase hex) is `<keyid>`. The root's own key answers here as well as at the legacy `tebako-key.asc` (which stays the human-facing spelling).
  - `https://www.tebako.org/.well-known/tebako-successors/<predecessor-fingerprint>.asc` — the `TEBAKO-ROOT-SUCCESSOR-V1` statement (§2's rotation format) rotating FROM `<predecessor-fingerprint>` (40 uppercase hex).

    The directory is a DIRECTORY (§9.3's keyserver rule): a fetched key authenticates by chaining, never by the channel that served it. `TEBAKO_ANCHOR_BASE` overrides the publication base (air-gapped mirrors, tests) — an AVAILABILITY knob, never a trust knob: admission still requires chaining to the embedded root.
- **The admission rule.** A retrieved key is usable — for this verification and for registration into the trusted keyring — iff:
  - **a.** its primary fingerprint IS the embedded root (`EMBEDDED_ROOT_FINGERPRINT` / `TEBAKO_TRUSTED_ROOT`); or

© 2026 The Tebako Project

b. it chains to a locally trusted root through verified successor statements: starting at the embedded root fingerprint, each hop fetches `tebako-successors/<current>.asc` and verifies the statement against the current root's key (the embedded public key, the trusted keyring, or a key retrieved and admitted earlier in the same walk); the walk ends at the retrieved key's fingerprint (admitted, basis = the verified path) or where no further statement exists (NOT admitted). The walk is bounded (8 hops) — a cycle is a dead end, not a hang; or

c. it is already in the trusted keyring (no fetch at all).

Anything else is the NAMED error `KeyRetrievalFailed` (exit 72, the trust-failure class): the message prints the retrieved key's fingerprint LOUDLY for out-of-band confirmation and names the manual path (`tebako keys import <file>`) — the same refusal shape as today, now with the fingerprint to compare. A `tebako-keys` URL that does not resolve (404) is the same named error class naming the URL.

- **Concurrency.** Retrieval is a tiny independent fetch and runs CONCURRENT with the plan's artifact streams (spec 05 §6); an artifact's signature verification starts when both its bytes and its key are present. A retrieved-and-admitted key is registered into the trusted keyring (§2's rotation rule: the loader TOFU-registers a verified successor) so the next verification is local.
- **The audit journal** records every retrieval: one row per event with the pinned keyid, the admitted fingerprint, the source URL, and the chain basis (`embedded-root` / `successor:<path>` / `already-trusted`) — trust events are exactly what the journal is for. Failed retrievals journal too (`event=key-retrieval-failed`).
- **`TEBAKO_OFFLINE=1` skips retrieval entirely** (no network): the verification keeps the pre-ceremony outcome — the pinned keyid is absent, the named untrusted-signer error (exit 72) stands.

© 2026 The Tebako Project